Mini-project 2 in Advanced Machine Learning (02460)

by Bashar Bdewi (S183356) on 2 April 2026

Part A: Pull-back geodesics

Variational autoencoders provide a flexible latent-variable model for nonlinear generative modeling (Tomczak, 2024). We trained a VAE with a standard Gaussian prior and Gaussian likelihood on a non-binarized MNIST subset containing the classes 0, 1, and 2, using 2048 observations and a two-dimensional latent space for visualization. During training, small Gaussian noise was added to the inputs as suggested in the handout. The geometric motivation is that Euclidean distances in the latent coordinates are not invariant under nonlinear reparameterizations, so distances should instead be induced from the decoder map (Hauberg, 2026).

Let f denote the decoder mean and let $c : [0, 1] \to \mathbb{R}^2$ be a latent curve. We approximated pull-back geodesics by minimizing the discrete curve energy

$$\tilde{E}[c] = \sum_{s=1}^{S} \|f(c_s) - f(c_{s-1})\|^2,$$

where $\{c_s\}_{s=0}^{S}$ are uniformly sampled points along the curve. In practice, c was parameterized as a third-order polynomial with fixed endpoints and optimized with Adam. This follows the numerical geodesic construction in the lecture notes, where energy minimization is preferred to direct length minimization because it is more stable and yields constant-speed geodesics in the continuous setting (Hauberg, 2026).

Figure 1 (left) shows latent posterior means together with 25 geodesics between fixed latent point pairs. The optimized curves are visibly different from straight latent interpolations and often bend toward dense latent regions. This indicates that the decoder-induced geometry captures structure not reflected by raw Euclidean coordinates. The single-decoder geodesics are already meaningful, but they still depend on the particular retraining, which motivates the ensemble extension.

Part B: Ensemble VAE geometry

To make the geometry more robust, we extended the model to an ensemble-decoder VAE and trained $M = 10$ independent retrainings with up to $D = 3$ decoders. For a latent curve c, the ensemble energy was approximated by

$$\tilde{E}[c] \approx \sum_{i=0}^{N-1} \mathbb{E}_{l,k} \left\| f_l(c(t_i)) - f_k(c(t_{i+1})) \right\|^2,$$

where f_l and f_k are decoder means drawn uniformly from the ensemble. This follows the uncertainty-aware, noisy-manifold viewpoint from the module: unreliable regions should receive a larger geometric penalty, leading to more stable distances across retrainings (Hauberg, 2026).

We reused the same fixed test-image pairs across all retrainings, encoded them using posterior means, and measured both Euclidean and geodesic distances for $D = 1, 2, 3$. For each point pair we computed the coefficient of variation

$$\mathrm{CoV} = \frac{\sigma(d^{(1)}, \ldots, d^{(M)})}{\mu(d^{(1)}, \ldots, d^{(M)})},$$

and then averaged across pairs. Figure 1 (right) shows ensemble geodesics, while Figure 2 reports the average CoV. The Euclidean distance remained almost unchanged across ensemble sizes, with average CoV around 0.196 for $D = 1, 2, 3$. In contrast, the geodesic distance was consistently more stable and improved with larger ensembles: the average CoV decreased from 0.168 for $D = 1$ to 0.138 for $D = 2$ and to 0.126 for $D = 3$. Hence the ensemble geodesic is both qualitatively different from straight interpolation and quantitatively more reliable than Euclidean distance.

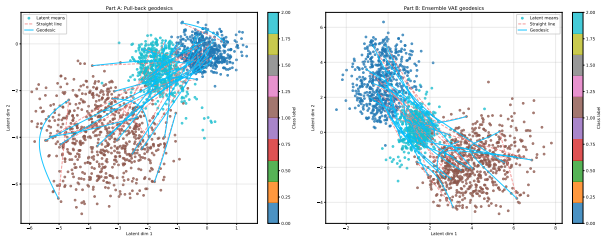

Figure 1: Geodesics between 25 fixed latent point pairs. Left: single-decoder VAE (Part A). Right: ensemble-decoder VAE (Part B).

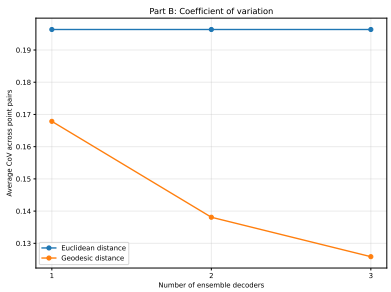

Figure 2: Average coefficient of variation as a function of the number of ensemble decoders. Geodesic distances are more stable than Euclidean distances and improve with larger ensembles.

Code snippets

The code below shows the core numerical ingredients used in the implementation: latent-curve energy under a single decoder, the ensemble energy for Part B, the geodesic optimizer, and the CoV summary used for the robustness comparison.

```
def curve_energy_mean_decoder(curve, model, num_points=20):
    z_points = latent_curve_points(curve, num_points, curve.start.device)
    x_means = model.decode_mean(z_points)
    deltas = x_means[1:] - x_means[:-1]
    sq = deltas.flatten(start_dim=1).pow(2).sum(dim=1)
    return (num_points - 1) * sq.sum()
```

```
def curve_energy_ensemble(curve, model, num_points=20,
                          num_mc_samples=1, num_decoders_to_use=3):
    z_points = latent_curve_points(curve, num_points, curve.start.device)
    decoded = [model.decode_mean(z_points, decoder_idx=d)
               for d in range(num_decoders_to_use)]
    energy = 0.0
    for s in range(num_points - 1):
        for _ in range(num_mc_samples):
            l = torch.randint(0, num_decoders_to_use, (1,)).item()
            k = torch.randint(0, num_decoders_to_use, (1,)).item()
            energy += ((decoded[k][s + 1] - decoded[l][s]) ** 2).sum() / num_mc_samples
    return (num_points - 1) * energy
```

```
def optimize_geodesic(curve, energy_fn, lr=0.1, epochs=600):
    optimizer = torch.optim.Adam(
        [p for p in curve.parameters() if p.requires_grad], lr=lr
    )
    for _ in range(epochs):
        optimizer.zero_grad()
        energy = energy_fn()
        energy.backward()
        optimizer.step()
    return curve
```

```
def compute_cov_summary(df):
    rows = []
    for d in sorted(df["num_decoders_used"].unique()):
        sub = df[df["num_decoders_used"] == d]
        euc = []
        geo = []
        for pair_id in sorted(sub["pair_id"].unique()):
            pair = sub[sub["pair_id"] == pair_id]
            euc.append(pair["euclidean_distance"].std() /
                       pair["euclidean_distance"].mean())
            geo.append(pair["geodesic_distance"].std() /
                       pair["geodesic_distance"].mean())
        rows.append((d, sum(euc) / len(euc), sum(geo) / len(geo)))
    return rows
```

Use of Generative AI

Generative AI was used as a support tool for code organization and debugging. All final implementation choices, experiments, results, and conclusions were checked by the author, who takes full responsibility for the submitted work.

References

Jakub M. Tomczak. *Deep Generative Modeling*. 2nd edition, Springer, 2024.

Søren Hauberg. *Latent Machine Learning Geometry*. Course lecture notes, Technical University of Denmark, 2026.